ELO320 - Estructuras de Datos y Algoritmos
Introducción a C
Arreglos en C

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

1er Semestre 2025

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Clase Activa: Módulo 1

Ejercicio 1.1

Cree una función que reciba como parámetro un arreglo de enteros y el tamaño de éste, y luego solicite al usuario esa cantidad de valores, necesarios, para rellenar este arreglo. Puede usar una de las dos siguientes declaraciones:

```
void add_datos(int arreglo[], int tam);

void add_datos(int * arreglo, int tam);
```

Aprendemos: Definir un arreglo, recorrer un arreglo, rellenar un arreglo. Paso por referencia.

Clase Activa: Módulo 1

Ejercicio 1.2

Cree una función que reciba como parámetro un arreglo de enteros y el tamaño de éste, y luego muestre por pantalla los valores almacenados en dicho arreglo. Puede usar una de las dos siguientes declaraciones:

```
void imprimir_datos(int arreglo[], int tam);

void imprimir_datos(int * arreglo, int tam);
```

Aprendemos: Definir un arreglo, recorrer un arreglo, rellenar un arreglo. Paso por referencia.

Clase Activa: Módulo 1

Ejercicio 1.3

Cree programa que declare un arreglo de enteros de un tamaño solicitado al usuario por pantalla. Luego, debe, en conjunto con el usuario, rellenar el arreglo y mostrar por pantalla su contenido. **Utilice las funciones creadas en los ejercicios 1.1 y 1.2**.

Aprendemos: Definir un arreglo, recorrer un arreglo, rellenar un arreglo. Paso por referencia.

Clase Activa: Módulo 2

Ejercicio 2.1

Cree y utilice una función que reciba como parámetro un arreglo de enteros y el tamaño de éste, y luego calcule el promedio entre todos los valores almacenados en dicho arreglo. Puede usar una de las dos siguientes declaraciones:

```
float promedio(int arreglo[], int tam);

float promedio(int * arreglo, int tam);
```

Aprendemos: Operar sobre un arreglo, recorrer un arreglo.

Clase Activa: Módulo 2

Ejercicio 2.2

Cree y utilice una función que reciba como parámetro un arreglo de enteros y el tamaño de éste, y luego calcule la desviación estándar entre todos los valores almacenados en dicho arreglo. Puede usar una de las dos siguientes declaraciones:

```
float desviacion(int arreglo[], int tam);

float desviacion(int * arreglo, int tam);
```

$$\sigma = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}}$$

Aprendemos: Operar sobre un arreglo, recorrer un arreglo.

Clase Activa: Módulo 2

Ejercicio 2.3

Cree y utilice una función que reciba como parámetro un arreglo de enteros y el tamaño de éste, y luego calcule la varianza entre todos los valores almacenados en dicho arreglo. Puede usar una de las dos siguientes declaraciones:

```
float varianza(int arreglo[], int tam);

float varianza(int * arreglo, int tam);
```

$$V = \sigma^2$$

Aprendemos: Operar sobre un arreglo, recorrer un arreglo.

Clase Activa: Módulo 2

Ejercicio 2.4

Cree y utilice una función que reciba como parámetro un arreglo de enteros y el tamaño de éste, y luego entregue el arreglo ordenado de menor a mayor en el arreglo original. Puede usar una de las dos siguientes declaraciones:

```
void sort(int arreglo[], int tam);

void sort(int * arreglo, int tam);
```

Aprendemos: Operar sobre un arreglo, recorrer un arreglo.

Clase Activa: Módulo 2

Ejercicio 2.5

Cree y utilice una función que reciba como parámetro un arreglo de enteros y el tamaño de éste, y luego entregue el menor valor del arreglo. Puede usar una de las dos siguientes declaraciones:

```
int menor(int arreglo[], int tam);

int menor(int * arreglo, int tam);
```

Aprendemos: Operar sobre un arreglo, recorrer un arreglo.

Clase Activa: Módulo 2

Ejercicio 2.6

Cree y utilice una función que reciba como parámetro un arreglo de enteros y el tamaño de éste, y luego entregue el mayor valor del arreglo. Puede usar una de las dos siguientes declaraciones:

```
int mayor(int arreglo[], int tam);

int mayor(int * arreglo, int tam);
```

Aprendemos: Operar sobre un arreglo, recorrer un arreglo.

Clase Activa: Módulo 3

Ejercicio 3.1

Cree un programa que solicite un tamaño m y n, y luego defina un arreglo bidimensional de enteros con dichos valores.

Aprendemos: Arreglo bidimensional.

Clase Activa: Módulo 3

Ejercicio 3.2

Cree una función que reciba como parámetros un arreglo bidimensional, y sus dimensiones m y n. La función debe permitir rellenar dicho arreglo. Puede usar una de las dos siguientes declaraciones:

```
void add_datos_2(int arreglo[][n], int m, int n);

void add_datos_2(int ** arreglo, int m, int n);
```

Aprendemos: Arreglo bidimensional. Recorrido bidimensional.

Clase Activa: Módulo 3

Ejercicio 3.3

Cree una función que reciba como parámetros dos arreglos bidimensionales de igual dimensión, y el valor de sus dimensiones m y n. Considerando que ambos arreglos pueden ser visto como matrices, la función debe calcular la suma de ambas de matrices y almacenar dicho resultado en alguna de las dos matrices pasadas como parámetros. Puede usar una de las dos siguientes declaraciones:

```
void suma(int m1[][], int m2[][n], int m, int n);

void suma(int ** m1, int ** m2, int m, int n);
```

Aprendemos: Arreglo bidimensional. Recorrido bidimensional.

Clase Activa: Módulo Desafío

Ejercicio Desafío

Cree un programa que solicite al usuario ingresar una palabra. El programa debe determinar si la palabra es palíndroma o no. Utilice funciones para hacer su trabajo más ordenado y sencillo. Recuerde que una palabra es palíndroma si se lee de igual forma de izquierda a derecha como de derecha a izquierda.

`reconocer` es palíndroma. `programar` no es palíndroma.

Aprendemos: Strings, Arreglo de Caracteres. Recorrido Inverso.

Información de Vital Importancia para la Clase Activa[1].

[1] Depende exclusivamente de uds. leerla y comprenderla.

Arreglos

- **Arreglo:** Conjunto homogéneo y ordenado de un tipo de datos.
- Se identifican por su posición relativa mediante un *índice* encerrado con paréntesis cuadrado ([], parte de índice 0).

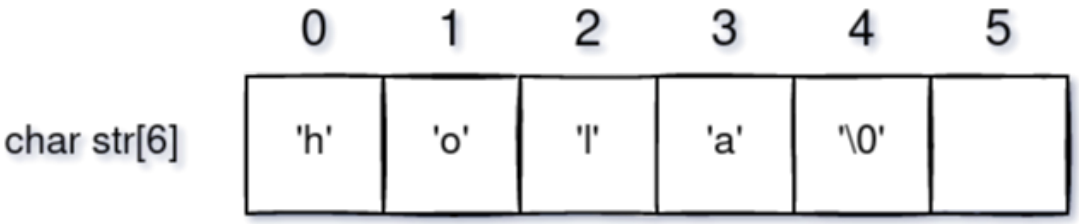
```
int arr[6]; //6 enteros creados en arr.
arr[3] = 222; //Al cuarto elemento de arr se le asigna un 222.
```

	0	1	2	3	4	5
int arr[6]	200	15	355	222	1765	55

Arreglos

- **String:** Estructura de datos para representar palabras.
- Arreglo de `char` y el caracter `'\0'`.

```
// Los string terminan con '\0' (fin de string)
// '\0' ocupa 1 char en el string.
char str[6] = "hola"; //String de largo máximo 5.
str[3] = 'i'; // ¿Qué hace?
printf("%s\n", str);
```



- Veremos más sobre strings las próximas clases.

Arreglos Multidimensionales e Inicialización

Además, se pueden definir arreglos multidimensionales.

También, los arreglos pueden ser inicializados utilizando { y }.

C

```
#include<stdio.h>

int main(){
        char nom[10] = "Camilo";
        char a[5][20] = {"Juan", "Francisco", "Pedro"};
        int b[3][3] = {{1,2,3}, {4,5,6}, {7,8,9}}, i, j;

        for(i=0;i<3;i++){
                for(j=0;j<3;j++) //¿Qué pasó con las llaves?
                        printf("%d ", b[i][j]);
                printf("\n");
        }
}
```

Recorrer un arreglo

Podemos utilizar cualquiera de las estructuras de control iterativas para recorrerlos.

C

```
#inclede<stdio.h>

int main(){
        int i;
        float a[5];

        for(i=0;i<5;i++){
                float val; //¿Cuántas veces nace y muere val?
                scanf("%f", &val);
                a[i]=val;
        } //¿Cómo reducir este código?

}
```